

Laboratorio S5: Mexi Banco con Compose

Alumno: Echeverria Goicochea Erick Isaac

Pasos 1 y 2: Clonar y levantar con Compose

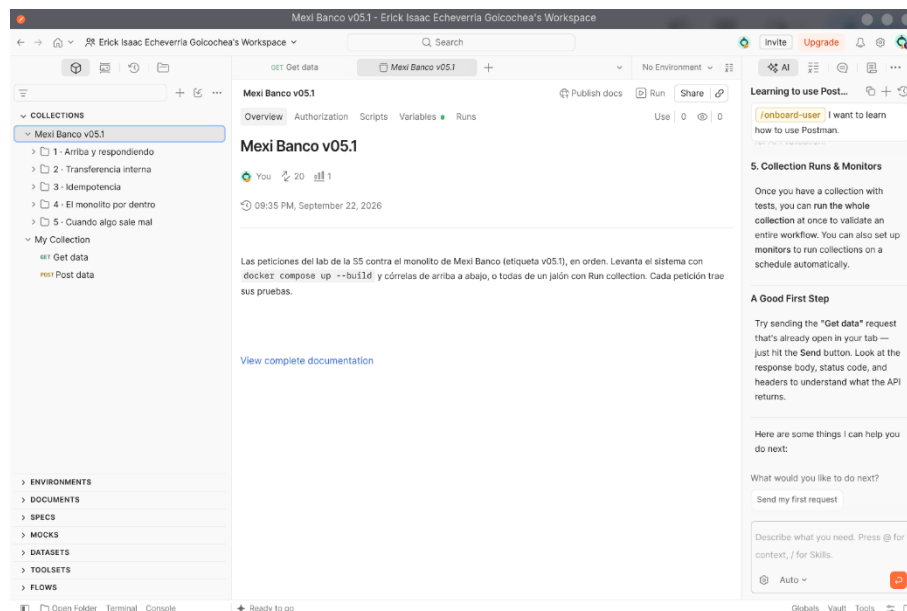
Después de clonar el repositorio, se levantó el entorno ejecutando `docker compose up --build -d`. Este comando compiló el proyecto y arrancó los contenedores en segundo plano.

```
(erick@pc-bd-eeg) [~/sd-2027-1/entregas/echeverria_erick/s05/mexi-banco]
$ docker compose up --build -d
[+] Building 135.8s (19/19) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 571B                                          0.0s
=> [internal] load build definition from Dockerfile                    0.0s
=> => transferring dockerfile: 440B                                     0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.2s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine 1.2s
=> [internal] load .dockerignore                                       0.0s
=> => transferring context: 2B                                          0.0s
=> [builder 1/8] FROM docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:cd87715a8d45cfaa42419207c64680234f62785c49055cccd20437b5c9018380 33.0s
=> => resolve docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:cd87715a8d45cfaa42419207c64680234f62785c49055cccd20437b5c9018380 0.0s
=> => sha256:cd87715a8d45cfaa42419207c64680234f62785c49055cccd20437b5c9018380 2.68kB / 2.68kB 0.0s
=> => sha256:a6a1f4589e27f44da02e788077573ed58a245b3ab3821e323f817d0a087 1.94kB / 1.94kB 0.0s
=> => sha256:1b383e9f9993c8ab07d06e1caf32cb7a16e7f9ae1171c3738da77aaa47d5991c 4.50kB / 4.50kB 0.0s
=> => sha256:2e2de96513ba9eb53b431787ec8a65cddc380ac4772a3e4c4b714dcfde2a102b5 3.85MB / 3.85MB 0.8s
=> => sha256:9cfeb5b8514c9dbbd1d00e50bd1cacea20a9b32df93487951602cae61289310f 21.44MB / 21.44MB 6.7s
=> => extracting sha256:9cfeb5b8514c9dbbd1d00e50bd1cacea20a9b32df93487951602cae61289310f 0.4s
=> => sha256:5c633b3ab448192e7018639ae3e164c60ef5340505fef64172078c965568f768 158.33MB / 158.33MB 31.0s
=> => sha256:cb159d650e75c7da3af4cd09d95f44bd3e7373ecf0a9f1d47cfd2a7a82b492d 129B / 129B 7.1s
=> => sha256:4cb434d94307643c6a23bb1d216d216f4c64599139f4b2e0840044236a705083 2.46kB / 2.46kB 7.3s
=> => extracting sha256:5c633b3ab448192e7018639ae3e164c60ef5340505fef64172078c965568f768 1.8s
=> => extracting sha256:cb159d650e75c7da3af4cd09d95f44bd3e7373ecf0a9f1d47cfd2a7a82b492d 0.0s
=> [internal] load build context                                       0.0s
=> => transferring context: 52.16kB                                     0.0s
=> [stage-1 2/3] WORKDIR /app                                         0.1s
=> [builder 2/8] WORKDIR /app                                         0.1s
=> [builder 3/8] COPY pom.xml .                                       0.0s
=> [builder 4/8] COPY mvnw .                                          0.0s
=> [builder 5/8] COPY .mvn .mvn                                       0.0s
=> [builder 6/8] RUN ./mvnw dependency:go-offline -B                  95.3s
=> [builder 7/8] COPY src ./src                                       0.0s
=> [builder 8/8] RUN ./mvnw clean package -DskipTests                5.8s
=> [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar       0.1s
=> exporting to image                                                0.1s
=> => exporting layers                                                0.1s
=> => writing image sha256:a8597363b982c23a64539448abf08b150811aaeadd122e3774e107c92dca73b 0.0s
=> => naming to docker.io/library/mexi-banco-app                     0.0s
=> => resolving provenance for metadata file                           0.0s
[+] Running 5/5
✓ mexi-banco-app Built 0.0s
✓ Network mexibanco Created 0.1s
✓ Volume mexi-banco-mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.8s
✓ Container mexi-banco-app-1 Started 6.0s
```

Para comprobar que todo funcionaba correctamente, se ejecutó `docker compose ps`. Como se observa en la captura, tanto la base de datos (`mexi-banco-db-1`) como la aplicación (`mexi-banco-app-1`) alcanzaron el estado `healthy`, confirmando que iniciaron sin errores y están listas para recibir peticiones.

```
(erick@pc-bd-eeg) [~/sd-2027-1/entregas/echeverria_erick/s05/mexi-banco]
$ docker compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED      STATUS      PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"      app        3 minutes ago Up 3 minutes (healthy) 0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db          3 minutes ago Up 3 minutes (healthy) 5432/tcp
```

Tras levantar la base de datos e instalar Postman, el primer paso consistió en importar la colección de pruebas en el cliente local de Postman.



2. Pruebas de los Endpoints

Carpeta 1: Arriba y respondiendo

Primero se comprobó que la aplicación estuviera en línea mediante el endpoint de salud de Spring Boot, el cual respondió correctamente con estado "UP" (200 OK). Posteriormente, se mandó la petición para abrir una cuenta de origen con un saldo inicial de \$1,000 y se hizo una consulta GET para confirmar que el registro fue exitoso en la base de datos.

The screenshot shows the Postman interface for a workspace named "Salud de la app - Erick Isaac Echeverría Golcochea's Workspace". The collection "Mexi Banco v05.1" is selected, and the endpoint "GET Salud de la app" is active. The request is a GET to "({baseUri}) /actuator/health". The response is a 200 OK status with a 5 ms response time and 241 B of data. The JSON body is:

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

The screenshot shows the Postman interface for the same workspace. The endpoint "POST Abrir cuenta origen (1000)" is active. The request is a POST to "({baseUri}) /cuentas". The response is a 201 Created status with a 175 ms response time and 230 B of data. The JSON body is:

```
{
  "id": 1,
  "clabe": "0802189798134883965",
  "titular": "Ana",
  "saldo": 1000
}
```

The screenshot shows the Postman interface for the same workspace. The endpoint "GET Consultar cuenta origen" is active. The request is a GET to "({baseUri}) /cuentas/ ({origen})". The response is a 200 OK status with a 10 ms response time and 233 B of data. The JSON body is:

```
{
  "id": 1,
  "clabe": "0802189798134883965",
  "titular": "Ana",
  "saldo": 1000.00
}
```

Carpeta 2: Transferencia interna

En esta etapa, se abrió una segunda cuenta (destino) con un saldo inicial de \$500. A continuación, se ejecutó un POST para transferir \$200 de la cuenta origen a la destino. Al consultar ambas cuentas de manera individual después de la operación, verificamos que los saldos se actualizaron de forma precisa (\$800 en origen y \$700 en destino), lo que demuestra que la lógica transaccional de la API funciona de manera consistente.

Abrir cuenta destino (500) - Erick Isaac Echeverría Golcochea's Workspace

POST {{baseUri}}/cuentas

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
201 Created - 32 ms - 230 B
```

```
{
  "id": 2,
  "clave": "012180790134803905",
  "titular": "Beto",
  "saldo": 500
}
```

Transferir 200 de origen a destino - Erick Isaac Echeverría Golcochea's Workspace

POST {{baseUri}}/transferencias

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
201 Created - 33 ms - 296 B
```

```
{
  "id": 1,
  "claveOrigen": "002180790134803905",
  "claveDestino": "012180790134803905",
  "monto": 200,
  "fecha": "2026-09-23T03:42:14.578997689Z"
}
```

Consultar origen (800) - Erick Isaac Echeverría Golcochea's Workspace

GET {{baseUri}}/cuentas/{{(origen)}}

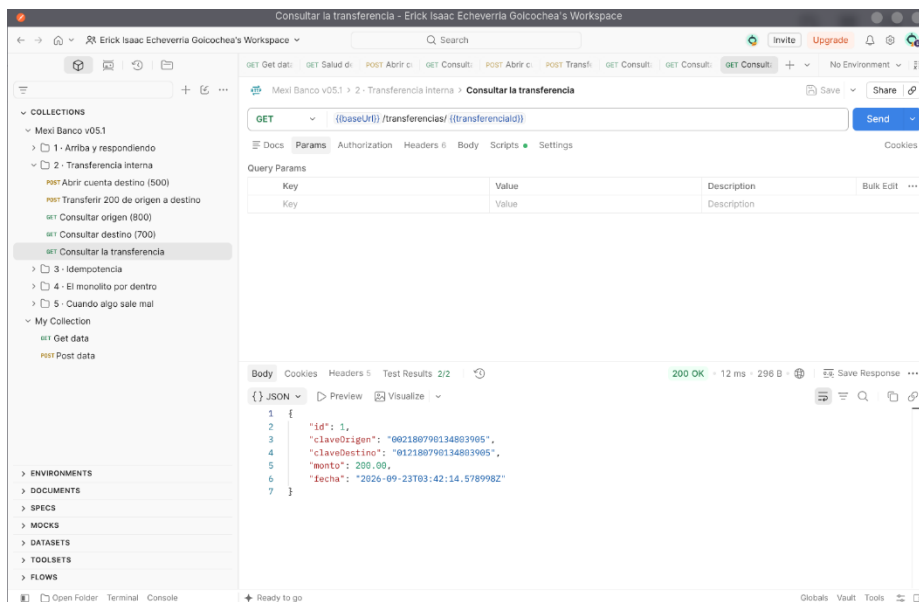
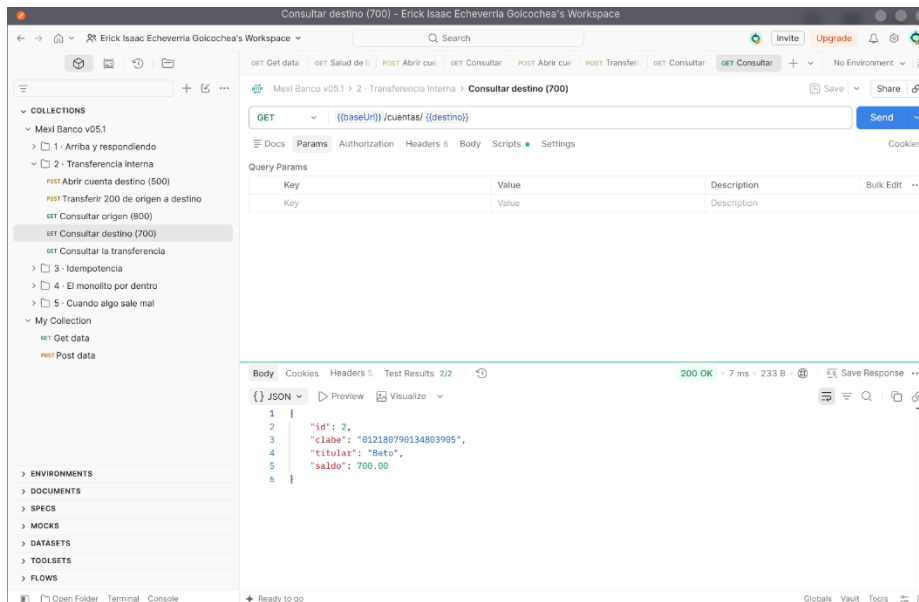
Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

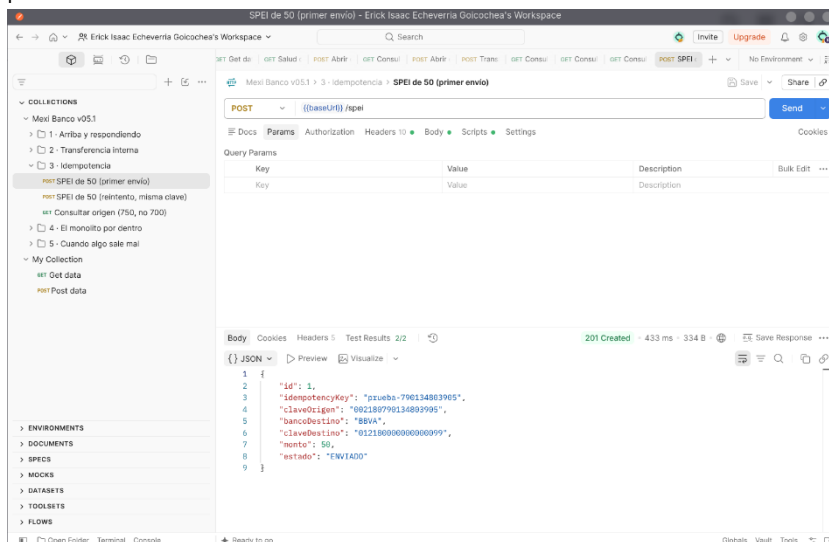
```
200 OK - 5 ms - 232 B
```

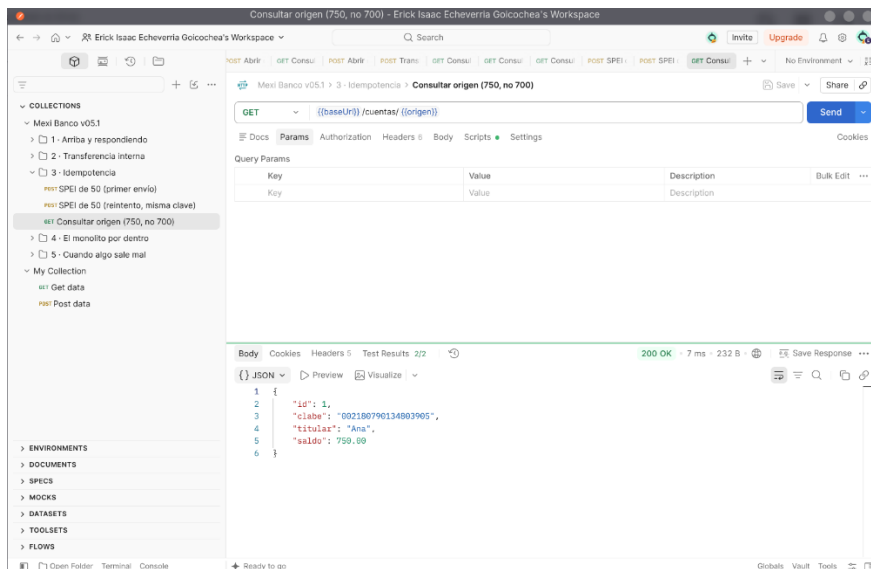
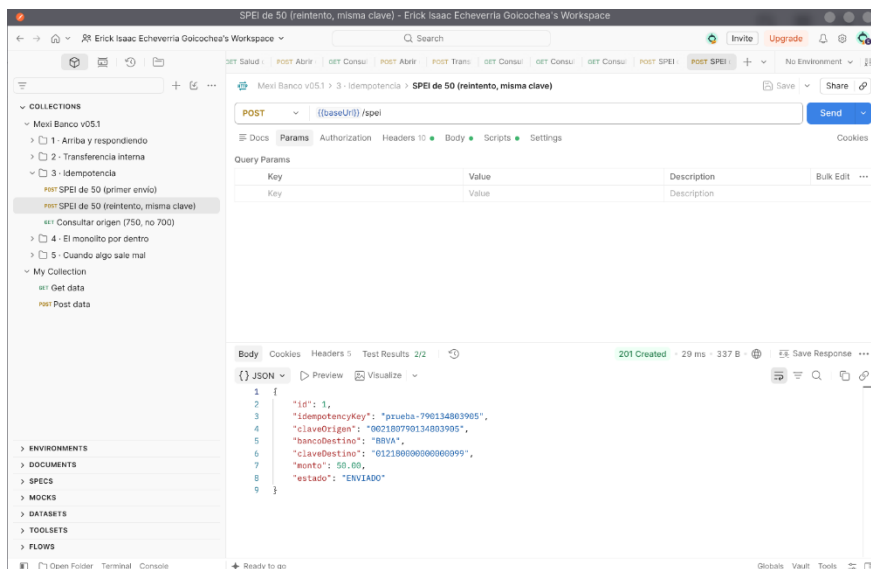
```
{
  "id": 1,
  "clave": "002180790134803905",
  "titular": "Ana",
  "saldo": 800.00
}
```



Carpeta 3: Idempotencia

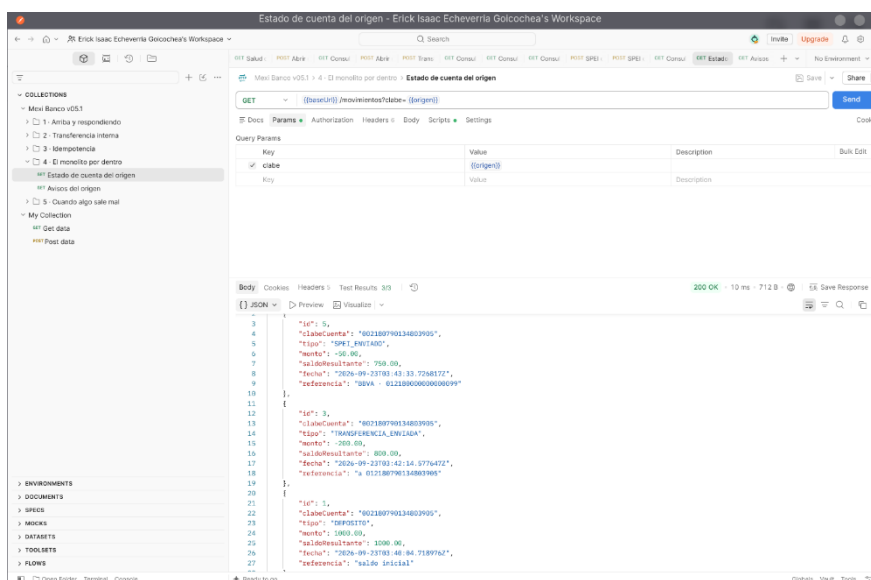
Se realizó una prueba de idempotencia. Se envía una orden SPEI por \$50 incluyendo una llave única (Idempotency-Key). Al reenviar la misma petición de forma idéntica, el sistema reconoció la llave, devolvió un 201 Created confirmando el estado de la operación, pero no volvió a restar dinero de la cuenta. Esto protege al usuario de cobros duplicados por problemas de red o clics dobles.

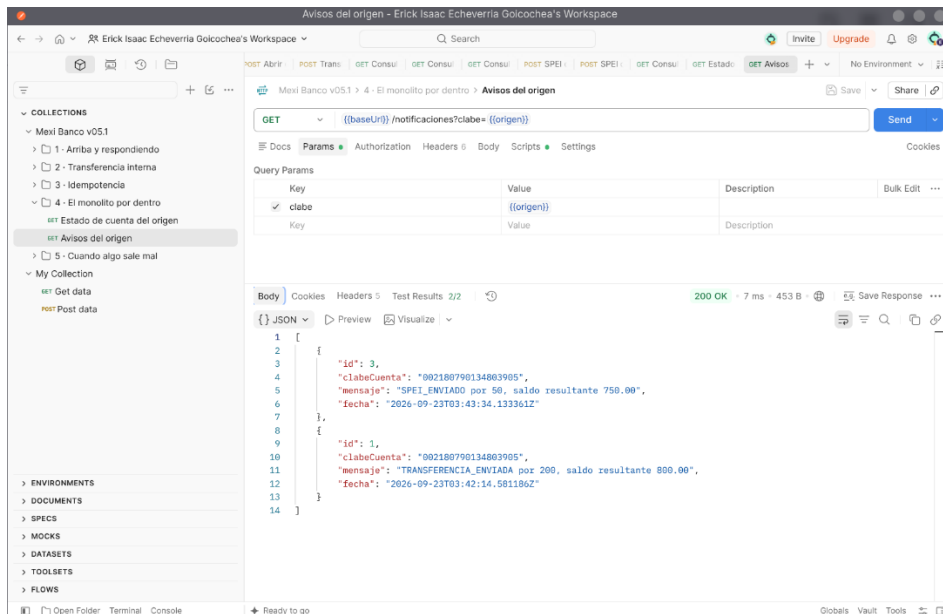




Carpeta 4: El monolito por dentro

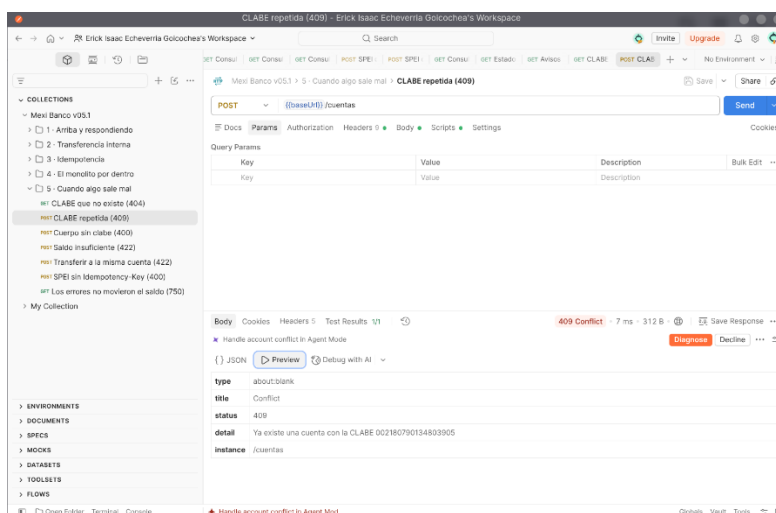
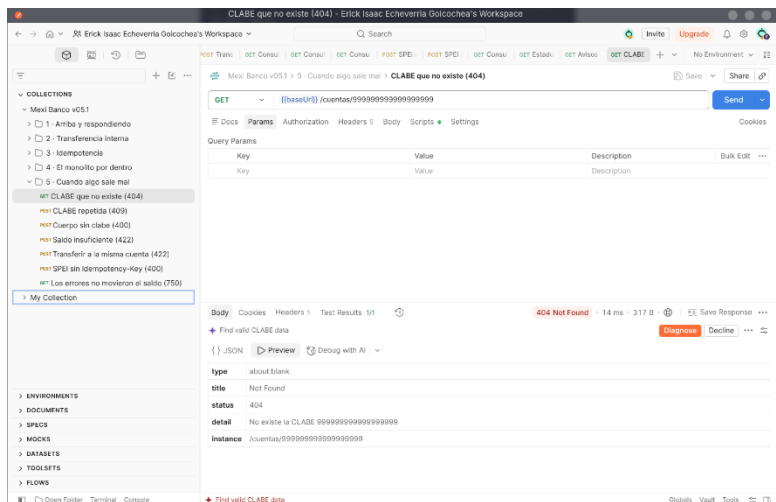
Consultamos el estado de cuenta y los avisos generados para la cuenta origen. El sistema nos devuelve un historial detallado en formato JSON con todos los movimientos registrados hasta el momento (el depósito inicial, la transferencia enviada y el pago SPEI), manteniendo la trazabilidad de las operaciones.





Carpeta 5: Cuando algo sale mal

Se enviaron peticiones diseñadas a propósito para fallar y así evaluar la solidez del manejo de errores. La API interceptó todas las fallas sin colapsar, devolviendo los códigos de estado HTTP correctos para cada situación: 404 cuando la cuenta no existe, 409 al intentar registrar una CLABE duplicada, 422 por errores de validación de negocio (saldo insuficiente o misma cuenta origen/destino), y 400 por una petición mal construida. Como comprobación final, revisamos que el saldo no se vio afectado por estos errores y se mantuvo intacto en \$750.



Cuerpo sin clabe (400) - Erick Isaac Echeverría Golcochea's Workspace

POST `{{baseUri}}/cuentas` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 4 Test Results 1/1 400 Bad Request - 31 ms - 233 B Save Response

+ Fix request body in Agent Mode Diagnose Decline

{ } JSON Preview Debug with AI

timestamp 2026-09-23T03:48:29.135+00:00

status 400

error Bad Request

path /cuentas

Fix request body in Agent Mode

Saldo insuficiente (422) - Erick Isaac Echeverría Golcochea's Workspace

POST `{{baseUri}}/transferencias` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 1/1 422 Unprocessable Entity (WebDAV) (RFC 4918) - 8 ms - 328 B Save Response

Thinking... Cancel

{ } JSON Preview Debug with AI

type about:blank

title Unprocessable Entity

status 422

detail Saldo insuficiente en la CLABE 002180790134803905

instance /transferencias

Thinking...

Transferir a la misma cuenta (422) - Erick Isaac Echeverría Golcochea's Workspace

POST `{{baseUri}}/transferencias` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 1/1 422 Unprocessable Entity (WebDAV) (RFC 4918) - 4 ms - 321 B Save Response

+ Fix transfer accounts payload Diagnose Decline

{ } JSON Preview Debug with AI

type about:blank

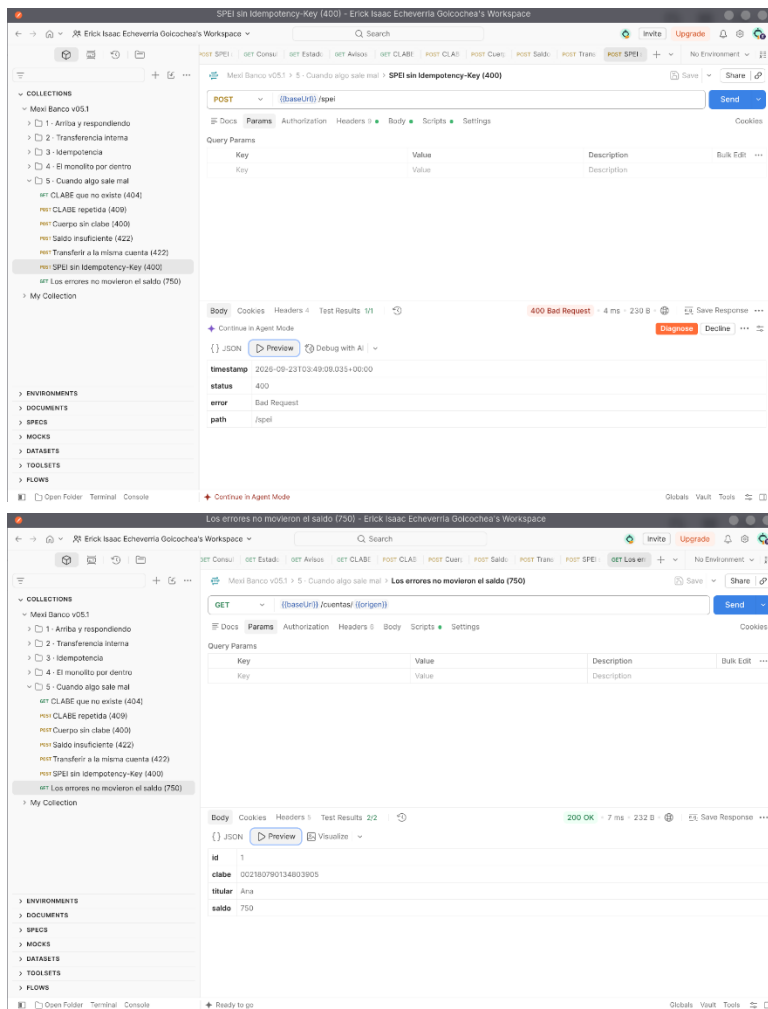
title Unprocessable Entity

status 422

detail La cuenta origen y la destino son la misma

instance /transferencias

+ Fix transfer accounts payload

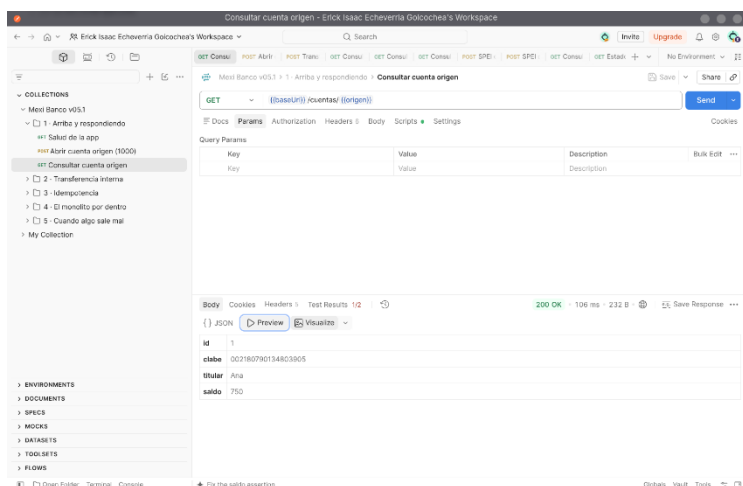


3. Prueba de Volúmenes y Persistencia

Paso 4: Conservación y pérdida de datos

Para comprender el ciclo de vida de los datos, apagamos los contenedores utilizando docker compose down y los levantamos nuevamente. Al hacer la consulta en Postman, los datos seguían ahí. El volumen que guarda la información de PostgreSQL no se destruye al apagar los contenedores, lo que permite mantener la persistencia de estado entre sesiones.

```
(erick@pc-bd-eeg)-[~/sd-2027-1/entregas/echeverria_erick/s05/mexi-banco]
$ docker compose down
docker compose up -d
[+] Running 3/3
  Container mexi-banco-app-1 Removed
  Container mexi-banco-db-1 Removed
  Network mexibanco Removed
[+] Running 3/3
  Network mexibanco Created
  Container mexi-banco-db-1 Healthy
  Container mexi-banco-app-1 Started
```

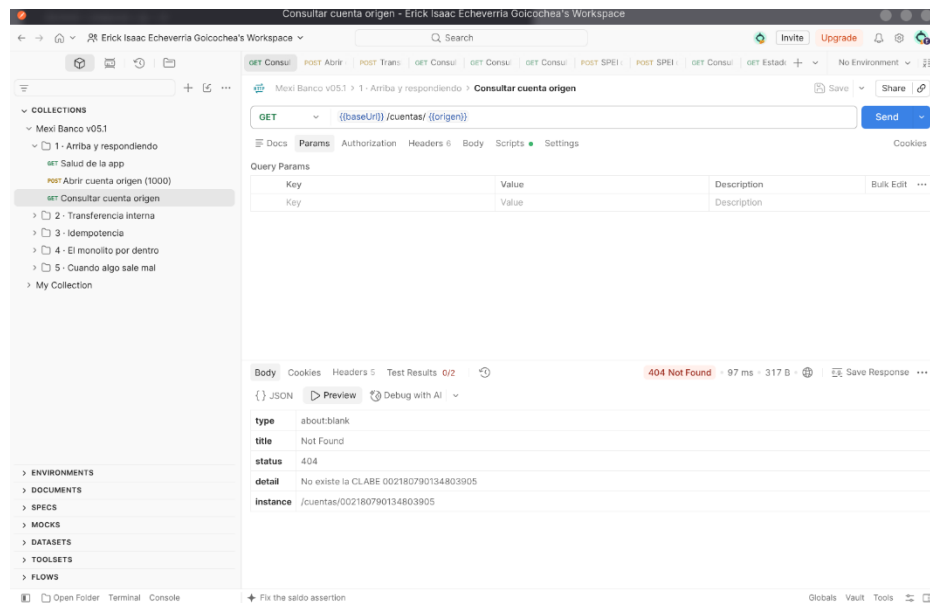


Por otro lado, cuando apagamos el entorno utilizando `docker compose down -v`, le indicamos a Docker que también remueva los volúmenes asociados. Al volver a levantar la aplicación y consultar la misma cuenta, obtuvimos un error 404 Not Found. Esto evidencia que el almacenamiento fue destruido y la base de datos se inicializó completamente limpia.

```

[erick@pc-bd-eeg] (~/.sd-2027-1/entregas/echeverria_erick/s05/mexi-banco)
$ docker compose down -v
docker compose up -d
[+] Running 4/4
  Container mexi-banco-app-1      Removed           0.3s
  Container mexi-banco-db-1       Removed           0.2s
  Volume mexi-banco_mexibanco_datos Removed           0.0s
  Network mexi-banco              Removed           0.4s
[+] Running 4/4
  Network mexi-banco              Created           0.1s
  Volume mexi-banco_mexibanco_datos Created           0.0s
  Container mexi-banco-db-1       Healthy          5.7s
  Container mexi-banco-app-1      Started          5.9s

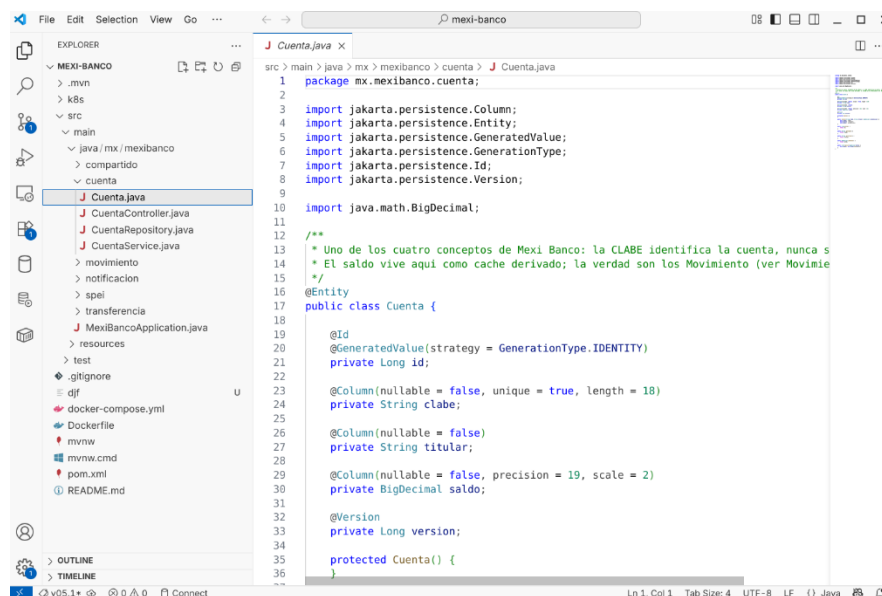
```



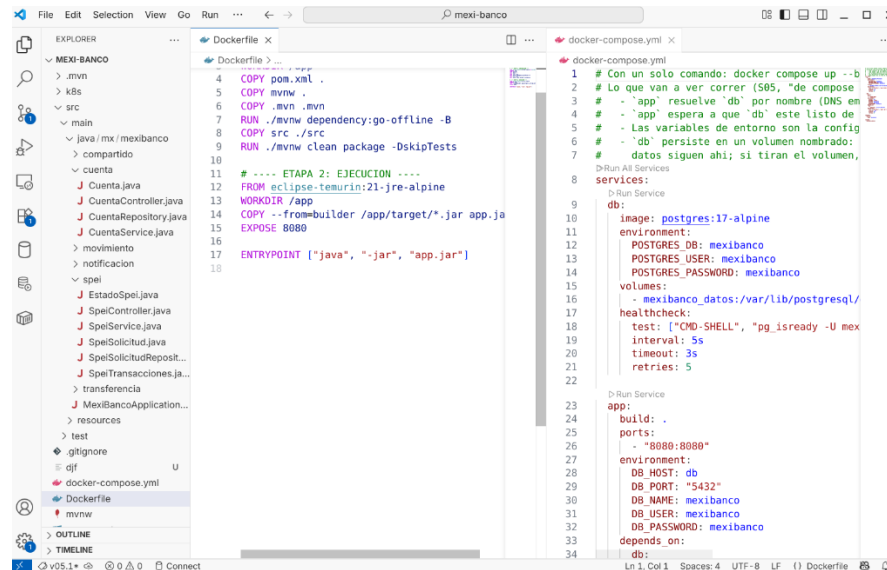
4. Análisis del Código e Infraestructura

Paso 5: Arquitectura del monolito

Al explorar el código fuente, se identificó la estructura del proyecto basada en una arquitectura de 4 capas. Cada módulo funcional (como la clase Cuenta) está estrictamente dividido en su propia Entidad, Repositorio, Servicio y Controlador, lo que facilita la lectura y el mantenimiento de la lógica de negocio del banco.



Por último, se revisó la infraestructura. El Dockerfile emplea un enfoque de compilación multi-etapa, donde una etapa pesada compila el código con Maven, y una segunda etapa más ligera simplemente ejecuta el .jar resultante. El archivo [docker-compose.yml](#) integra toda la aplicación, definiendo el volumen local para la base de datos, la red compartida y configurando los healthchecks para asegurar que la app espere a que Postgres esté completamente listo antes de recibir tráfico.



The screenshot shows an IDE with two files open: `Dockerfile` and `docker-compose.yml`. The `Dockerfile` is divided into two stages. Stage 1 (FROM eclipse-temurin:21-jre-alpine) copies the `pom.xml`, runs `mvnw` to set up dependencies, and copies the source code. Stage 2 (FROM eclipse-temurin:21-jre-alpine) copies the compiled `app.jar` and sets the entrypoint to run `java -jar app.jar`. The `docker-compose.yml` file defines two services: `db` (PostgreSQL) and `app` (the Java application). The `db` service uses the `postgres:17-alpine` image, sets environment variables for `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD`, and mounts a local volume `mexibanco_datos`. It also includes a healthcheck. The `app` service uses the `build` context, sets `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD` environment variables, and depends on the `db` service.

```
4 COPY pom.xml .
5 COPY mvnw .
6 COPY .mvn .
7 RUN ./mvnw dependency:go-offline -B
8 COPY src ./src
9 RUN ./mvnw clean package -DskipTests
10
11 # ---- ETAPA 2: EJECUCION ----
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 COPY --from=builder /app/target/*.jar app.jar
15 EXPOSE 8080
16
17 ENTRYPOINT ["java", "-jar", "app.jar"]
18
```

```
1 # Con un solo comando: docker compose up --b
2 # Lo que van a ver correr (S05, "de compose
3 # - 'app' resuelve 'db' por nombre (DNS em
4 # - 'app' espera a que 'db' este listo de
5 # - Las variables de entorno son la config
6 # - 'db' persiste en un volumen nombrado:
7 # datos siguen ahí; si tiran el volumen,
8
9 services:
10   db:
11     image: postgres:17-alpine
12     environment:
13       POSTGRES_DB: mexibanco
14       POSTGRES_USER: mexibanco
15       POSTGRES_PASSWORD: mexibanco
16     volumes:
17       - mexibanco_datos:/var/lib/postgresql/
18     healthcheck:
19       test: ["CMD-SHELL", "pg_isready -U mex
20       interval: 5s
21       timeout: 3s
22       retries: 5
23
24   app:
25     build: .
26     ports:
27       - "8080:8080"
28     environment:
29       DB_HOST: db
30       DB_PORT: "5432"
31       DB_NAME: mexibanco
32       DB_USER: mexibanco
33       DB_PASSWORD: mexibanco
34     depends_on:
35       db:
```